2. Manipulación e Interactividad de interfaces gráficas y naturales

Eduardo Delgado
DDINT – 2 DAM

Interfaz Usuario en [GUI] y [NUI]

La interfaz de usuario es el *Frontend o lado de cliente* de una aplicación

El *backend gestiona los datos y la lógica interna*, pero el usuario evalúa la app por su experiencia (UX) en el frontend.

Los *lenguajes de scripting* permiten *dar instrucciones al navegador o a una app* para ejecutar acciones directamente en el cliente.

Ejemplos de lenguajes de script Frontend: JavaScript | jQuery | React | Vue

Javascript

- Nació en los años 90 para ejecutarse en el navegador del usuario.
- Su función inicial fue validar formularios HTML antes de enviarlos al servidor.
- Esto permitía ahorrar tiempo y recursos, evitando la comunicación constante con el servidor.
- Hoy en día, *JavaScript* es la *base del desarrollo web moderno y la puerta de entrada a frameworks* como *React* o *Vue*.

Javascript: BOM

- **El Modelo de Objetos del Navegador o Browser Object Model [BOM] representa el entorno del navegador donde se ejecuta javascript**

  Permite controlar:

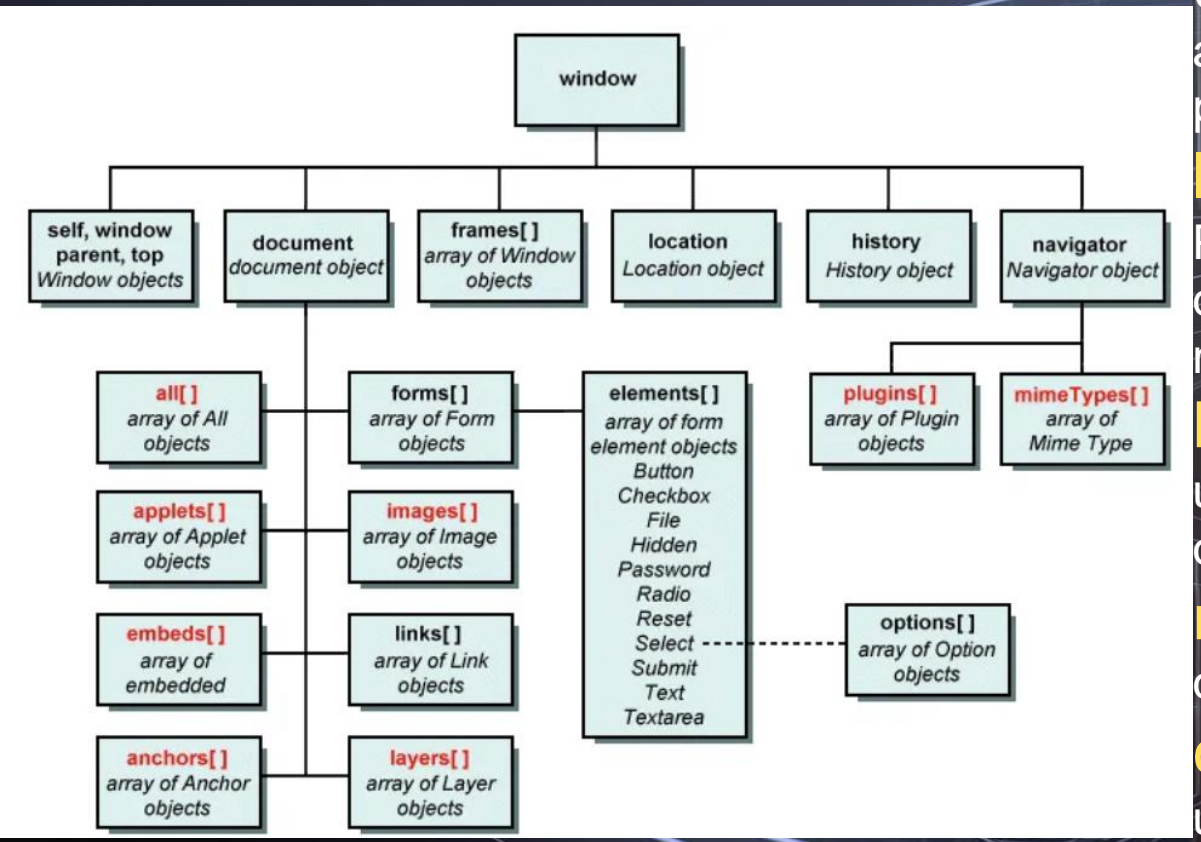
  - ***Ventanas*** window, open, close
  - ***Navegación*** location, history
  - ***Información del dispositivo*** navigator
  - ***Diálogos*** alert, prompt, confirm

JS ejemploBOM.js

```
1   window.alert("Hola!");
2   console.log(window.location.href);
```

El BOM maneja la “interfaz del navegador” pero no la página en sí

Javascript: BOM. Elementos y jerarquía



window - representa la ventana completa del navegador. Con window puedes manipular y controlar la ventana con acciones como mover, cambiar tamaño, abrir y cerrar pestañas.

location - contiene información sobre la página actual. Permite acceder a detalles como la URL completa, el nombre de dominio y el protocolo utilizado. Con location se puede redirigir a otras páginas y manipular la historia del navegador.

history - Proporciona acceso al historial de navegación del usuario, es decir, las páginas visitadas en la pestaña o marco de la página actual.

navigator - Almacena información sobre el navegador web que utiliza el usuario.

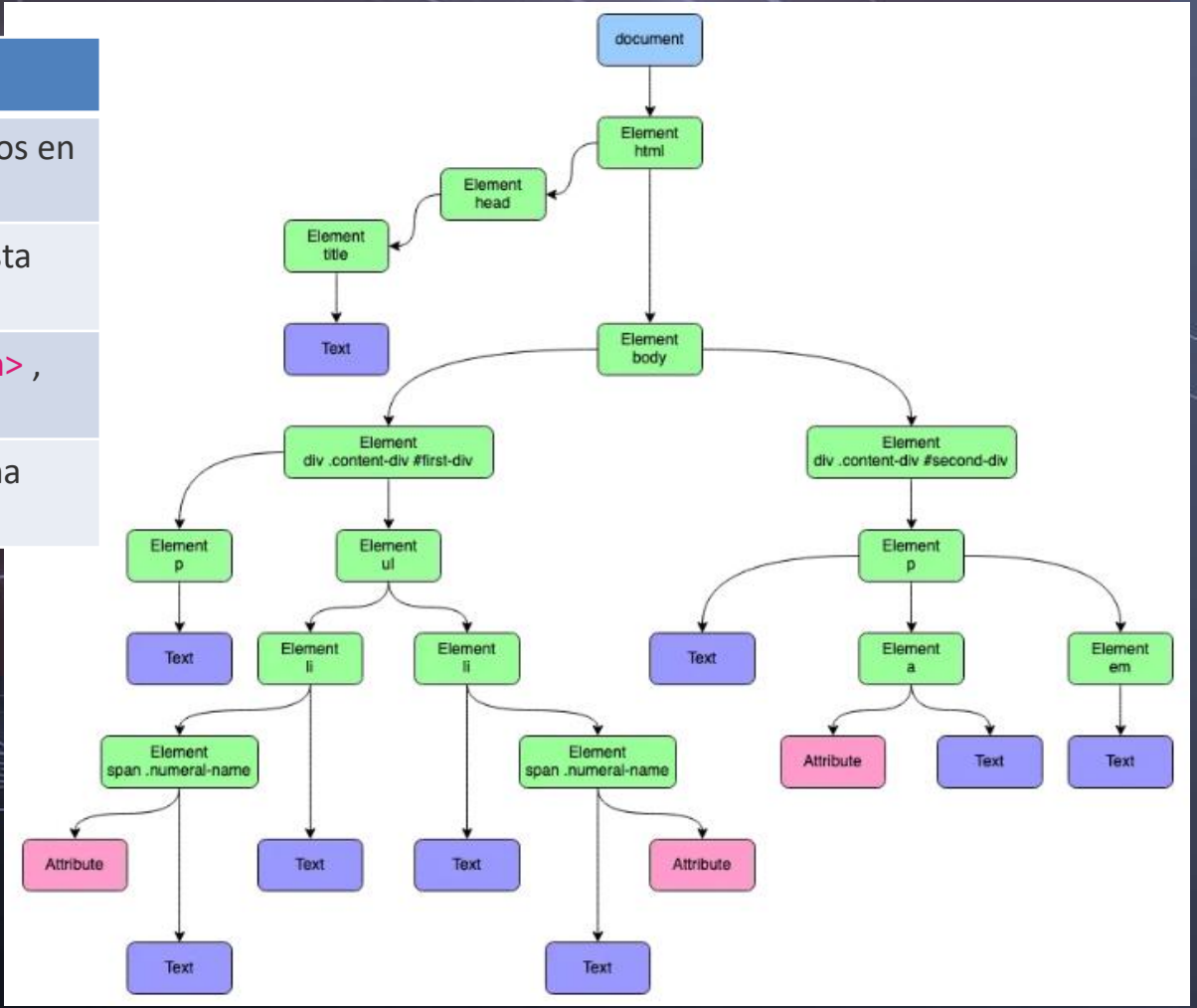
console - Proporciona métodos para la depuración. El más utilizado el `console.log()` que mostrará un mensaje a la consola.

document - Contiene todos los elementos de la página. Representa el DOM.

Javascript: DOM

- **El Modelo de Objetos del Documento o Document Object Model [DOM] es la interfaz de programación que representa el documento web. Se estructura por nodos de objetos organizados de forma jerárquica (árbol)**

Objeto del DOM	Descripción
Node	Representar elementos HTML, pero también otros elementos en un documento HTML, como texto o comentarios
NodeList	Representa una colección de nodos, proporcionando una lista ordenada y accesible de elementos.
Element	representa un elemento HTML concreto, como , , entre otros.
HTMLCollection	representa una colección de elementos, proporcionando una estructura similar a un array de elementosHTML



Javascript: DOM. Propiedades de document

Propiedad	Descripción	Ver en la Consola
docType	Devuelve el objeto definido por la etiqueta <!DOCTYPE>	console.log("Objeto doctype:",document.doctype);
head	Obtiene el objeto que representa la cabecera de la página, es decir, la sección <head>	console.log("Objeto head:",document.head);
body	Devuelve el objeto que representa el cuerpo de la página,correspondiente a la etiqueta <body>	console.log("Objeto body:",document.body);
title	Obtiene el título del documento que aparece en la etiqueta <title>	console.log("Título del documento:",document.title);
cookie	Devuelve en forma de cadena los valores de todas las cookies almacenadas y asociadas al documento, separados por punto y coma.	console.log("Cookies del documento:",document.cookie);
forms	Devuelve un objeto HTMLCollection de los formularios <forms> presentes en el document.	console.log(”Formularios en eldocumento:", document.forms);
links	Devuelve un objeto HTMLCollection de los hipervínculos <a> en el documento	console.log("Enlaces en el documento:",document.links);
images	Devuelve un objeto HTMLCollection de las imágenes <img> presentes en el documento.	console.log("Imágenes en el documento:",document.images);

Javascript: Sintaxis básica

- **Un script de JavaScript incluye *lista de sentencias* [instrucciones] de programación.**
- **En HTML, los programas JavaScript son *ejecutados por el navegador* web.**
- **Las *sentencias* de JavaScript están *compuestas de:*** valores, operadores, expresiones, palabras clave y comentarios

Comentarios

Los comentarios en JavaScript utilizan la misma sintaxis que en el lenguaje C. Se pueden usar r 2 tipos de comentarios:

- **Comentarios de línea: JavaScript ignorará cualquier texto entre // y el final de la línea (no se ejecutará).**
- **Comentarios multilínea: JavaScript ignorará cualquier texto entre /* y */ .**

.js sintaxis básica: Variables

- **En Javascript podemos declarar las *variables de 4 maneras diferentes:***

Variables

- **Automáticamente: declaroVariable = valor ;**
- ***var* : la variable puede ser utilizada desde el momento de su declaración hasta el final del código.**
- ***let* : la variable sólo podrá ser usada en el bloque de código dónde se declaró.**
- ***const*: la variable tiene un valor constante, cuando se le asigne un valor por primera vez, este ya no podrá ser modificado.**

.js sintaxis básica: Variables Ejemplo

ejemplo1js.html

```html
<!DOCTYPE html>
<html>
<head>
<script>
// Declaramos automáticamente la variable "marcaCoche" y la asignamos el valor "MG"
marcaCoche = "MG";

// Iniciamos una sentencia condicional "if"
if (marcaCoche == "MG" ) {

/* Declaramos 2 variables en su bloque de código:
Una con "var" que podemos utilizar fuera del código del "if */
var modeloCoche = "ZS" ;

// Otra con "let" que NO podemos utilizar fuera del código del "if"
let versionCoche = 'luxury' ;

// Mostramos por consola sus valores DENTRO del IF
console.log ("Dentro del IF");
console.log ("Marca" , marcaCoche);
console.log ("Modelo", modeloCoche);
console.log ("Versión", versionCoche);
}
/* Al intentar mostrar por consola el valor de la variable "versionCoche"
FUERA del IF indica que esta no tiene valor asignado ("undefined") ya que
no "ve" la variable declarada del "let" a la que asignamos un valor*/

console.log("Fuera del IF");
console.log ("Marca" , marcaCoche);
console.log ("Modelo", modeloCoche);
console.log ("Versión", versionCoche);
console.log ("El " + marcaCoche + modeloCoche + "tiene varias versiones," +
" pero no puedo ver cuál seleccionaste " + versionCoche);
</script>
</head>
<body>
<h2>Demo JavaScript en &lt; Head &gt; </h2>
<p>Abre la consola del navegador para ver el resultado</p>
</body>
</html>
```

.js sintaxis básica: Variables. Tipos de Datos

En JavaScript podemos guardar muchos tipos de datos dentro de variables:

- ***Number*** (numéricos)
- ***String*** (texto)
- ***Boolean*** (verdadero/falso)
- ***Object*** (objetos)
- ***Array*** (listas de valores, que también son objetos)
- ***Date*** (fechas)

.js sintaxis básica: Variables. Tipos de Datos

tiposdatosjs.html > ...

```html
<!DOCTYPE html>
<html>
<head>
<script>
// NUMÉRICOS
let length = 16;
let weight = 7.5;   // también números decimales

// CADENAS DE TEXTO (String)

let color = "Yellow";
let lastName = 'Johnson';   // sirven tanto comillas simples como dobles

// BOOLEANOS
let x = true;
let y = false;

// OBJETOS
const person = {
firstName: "John",
lastName: "Doe"
};

// ARRAYS
const cars = ["Saab", "Volvo", "BMW"];

// FECHAS
const date = new Date("2022-03-25");
</script>
</head>
</html>
```

.js sintaxis básica: Operadores Aritméticos

```
// Sumar dos números +
let operacion = 2 + 2;
console.log("Suma de 2 + 2 = "+operacion);

// Restar dos números -
operacion = 4 - 2;
console.log("Resta de 4 - 2 = "+operacion);

// Multiplicar dos números *
operacion = 4 * 4;
console.log("Multiplicar 4 x 4 = "+operacion);

// Dividir dos números /
operacion = 5 / 3;
console.log("División de 5 / 3 = "+operacion);

// Modulo de la división de 2 números (Nos devuelve el resto de una
división)
operacion = 5 % 3;
console.log("El resto de la división de 5 / 3 es: "+operacion);
```

.js sintaxis básica: Operadores Aritméticos

```
// Incrementar un número ++

console.log("Al incrementar el resultado anterior nos daría:"+(++operacion));
console.log("Si el operador de incremento lo pongo como sufijo, primero me devuelve el
valor: "+(operacion++)+" y luego realizaría el incremento quedando así: "+operacion);

// El decremento funcionaría de manera similar, utilizando "operacion--" o "--
operacion"

// Todos estos operandos los puedo añadir como prefijo de una asignación quedando así

let operacionAsignacion = 2; operacionAsignacion += 3;operacion = 2 + 3;

console.log("Si x = 2, y = 3")
console.log("Sería lo mismo utilizar 'x += y' ("+operacionAsignacion+")que hacerlo así
'x = x + y' ("+operacion+")");

// Así con todos los operandos.
```

.js sintaxis básica: Operadores de Comparación

```
// Comparamos valor y tipo de dato

let comparacion = ( 2 === 2);
console.log("Como no sabemos inicialmente el tipo de datos que contiene una variable en JavaScript");
console.log("Si queremos sabe si el contenido de una variable tiene el mismo valor y es del mismo tipo, utilizaremos el operador ' === '");
console.log("Como por ejemplo comprobar si '2===2' siendo ambos números enteros, esto devolvería:"+comparacion );

// Comparamos solo valor
comparacion = ( 2 == "2");
console.log("Si sólo quiero comprobar que ambos valores son iguales utilizaré el operador ' == '");
console.log("Como por ejemplo comprobar si (2=="2") siendo un número entero y un string, esto devolvería:" +comparacion );
```

.js sintaxis básica: Operadores de Comparación

```
// Comparamos desigualdad

console.log("De la misma forma sucede con la desigualdad usando'!==' para valor y tipo,
y '!=' sólo para valor");
let comparacion = ( 2 !== 3);
console.log("Como por ejemplo comprobar si'2!==3':"+comparacion );
comparacion = ( 2 != 3);
console.log("Y como por ejemplo comprobar si'2!=3':"+comparacion );

// Otros operandos de comparación

< → menor que
> → mayor que
<= → menor o igual que
>= → mayor o igual que
```

.js sintaxis básica: Operadores de Comparación

```
// Un caso especial es el operador ternario (?:)

console.log("Un caso especial es el operador ternario con el
formato ' (condición)? valorSiEsTrue : valorSiEsFalse ');
comparacion = (true)?"verdadero":"falso";
console.log("Como por ejemplo comprobar qué devuelve la consola
si (true)?"verdadero":"falso": "+comparacion );
```

.js sintaxis básica: Operadores lógicos

```
// Operadores lógicos que responden a la combinación de diferentes valores "Booleanos"
// AND Lógico
let comparacion = (true && true );
console.log("Si queremos comprobar que dos condiciones son ambas verdades utilizaremos
el operador '&&' (AND lógico)");
console.log("Como por ejemplo comprobar ('true' && 'true') que devolvería devolver:
"+comparacion);
console.log("NOTA: Cualquier otra combinación de valores booleanos con el AND lógico
devolvería FALSE");
// OR Lógico
Comparacion = (false || false );
console.log( "Si dos condiciones son ambas y también por separado FALSAS utilizaremos
el operador '||' (OR lógico)");
console.log('Como por ejemplo comprobar ("false" || "false") que devolvería devolver: '
+comparacion);
console.log("NOTA: Cualquier otra combinación de valores booleanos con el OR lógico
devolvería TRUE");
```

.js sintaxis básica: Operadores lógicos

```
// Operadores lógicos que responden a la combinación de diferentes valores "Booleanos"
// NOT lógico
Comparacion =! true;
console.log("Si queremos invertir el valor de una comparación a su valor complementario
(pasar el TRUE a FALSE) y viceversa utilizaremos el operador '!' (NOT lógico)");
console.log('Como por ejemplo comprobar (!"true") que devolvería devolver: '
+comparacion);
```

.js sintaxis básica: Operadores de tipo

```
// El operador "typeof" devolverá el tipo de dato del valor que contiene la variable
let variable = "Hola";
console.log("El operador 'typeof' devolverá el tipo de dato del valor que contiene la
variable");
console.log("Por ejemplo el valor de la variable llamada 'variable' es :"
+variable);
console.log("Y al llamar a 'typeof(variable)' me indica que es del tipo: "
+typeof(variable));
// El operador "instanceof" devolverá verdadero si un objeto es una instancia de un
tipo de objeto
console.log("El operador 'instanceof' devolverá verdadero si un objeto es una instancia
de un tipo de objeto");
let fecha = new Date();
console.log('Por ejemplo al llamar a (fecha instanceof Date) devolverá: '
+(fecha instanceof Date));
```

.js sintaxis básica: Operadores de string

```
// El + también se puede usar para agregar cadenas (concatenar)
let variable = "Hola";
let variable2 = " ¿qué tal?";
console.log(variable + variable2);
// Y si uno de los dos operandos del + es una cadena, seguirá concatenando la cadena al
valor del otro operando
let numero = 2;
console.log("Sumando " + numero + " y " + numero + ". ¿Cuánto da?" );
// Un String se puede crear como primitivas, a partir de literales de cadena o como
objetos, ó usando el constructor String ():
let texto = new String ("Un objeto String");
// Esto permite que cualquier String (literales u objetos) puedan utilizar los métodos
y propiedades de esta clase.
console.log(variable + " ¿contiene la letra \"a\"? " +(variable.includes('a')));
console.log("¿Cuántos caracteres tiene el texto \""+ variable2 + "\"? "
+(variable2.length));
```

.js sintaxis básica: Condiciones

```
// Utilizaremos una sentencia "if" para especificar un bloque de código// que se debe
ejecutar cuándo la condición sea verdadera
if(true){
console.log("La condición del \"if\" fue verdadera.");}
// Use la instrucción "else" para especificar un bloque de código que se // ejecutará
si la condición es falsa.
if(false) {
console.log("Este mensaje no se mostrará nunca al ser la condición falsa.");}
else{
Console.log("Se muestra este mensaje porque la condición del \"if\" fue falsa y este
mensaje está dentro del \"else\"");}
// Use la instrucción "else if" para especificar una nueva condición si la primera
condición es falsa.
let condicion1 = false, condicion2 = false;
if(condicion1) {
console.log("Este mensaje no se mostrará nunca al ser la 1ª condición falsa.");}
else if(condicion2) {
console.log("Este mensaje no se mostrará nunca al ser la 2ª condición falsa.");}
else{
console.log("Se muestra este mensaje al estar en el último \"else\" que llegó por ser
las condiciones previas falsas.");}
```

.js sintaxis básica: Condiciones

```
// Use la instrucción "Switch" para seleccionar uno de los muchos bloques de código que se ejecutará.
// NOTA: Para que una vez que encuentre la coincidencia de valor, no sigua ejecutando el resto de instrucciones
// que le siguen, aunque sean de otros valores que no coincidan, hay que poner la sentencia "break;"
// El método "getDay()" de un objeto "Date" devuelve el día de la semana como un número entre 0 y 6.
switch(new Date().getDay()) {
        case 0:
                dia = "Domingo";
                break;
        case 1:
                dia = "Lunes";
                break;
        case 2:
                dia = "Martes";
                break;
        case 3:
                dia = "Miércoles";
                break;
        case 4:
                dia = "Jueves";
                break;
        case 5:
                dia = "Viernes";
                break;
        case 6:
                dia = "Sábado";
                break;
```

.js sintaxis básica: Bucles → while

```
// El bucle "while" ejecutará su bloque de código después de comprobar que su condición es "true"
// Y una vez ejecutado su bloque de código repetira el proceso, comprueba su condición, y si ésta
// es "true" volverá a ejecutar su bloque de código. Este proceso se repetirá, hasta que la
condición
// devuelva "false" o dentro del código se ejecute la sentencia "break;"
console.log("Mostraremos los números, pares del 1 al 10:");
let numero =1;
while(numero <= 10){
        if ((numero % 2) == 0) {
                console.log("Número "+numero); }
        numero ++;}
// El bucle "do while" es una variante del "while", actua exactamente igual, salvo que la
//comprobación de su  condición se realizará trás ejecutar el bloque de código.
console.log ("Mostraremos los números, impares del 1 al 10:");
numero =1;
do {
        if ((numero % 2) == 0) {
                console.log("Número "+numero); }
        numero ++;
} while(numero <= 10);
```

.js sintaxis básica: Bucles → for

```
// El bucle "for" lo utilizaremos para repetir un bloque de código un número determinado de veces
console.log ("Mostraremos los números, múltiplos de 3 del 1 al 10");
for(let i = 1;i<=10;i++){
        if ((numero % 3) == 0) {
                console.log("Número "+numero); }
}
// Podemos utilizar el bucle "for in" para:
// 1.- Recorrer las propiedades de un objeto
const persona = {nombre:"Juan", apellido:"Nadie",edad:25};
let texto = "";
for (let x in persona) {
        texto += persona[x];
        texto += " ";}
console.log (texto);
// 2.- Recorrer los elementos de un array
const numerosPrimitiva = [1,10,18,21,30,3];
let numeros = "";
for (let x in numerosPrimitiva) {
        numeros += numerosPrimitiva[x];
        numeros += " ";}
// String.substring(posicionCaracterInicio, PosicionCaracterFin)
// el carácter posicionado en "PosiionCaracterFin" no se incluirá en la subcadena generada.
console.log( "Mi apuesta de la primitiva es: "+numeros.substring(0,(numeros.length-1)));
```

.js sintaxis básica: funciones

```
// Una función JavaScript es un bloque de código diseñado para realizar
//una tarea particular.
function cuadrado(numero){
// El operador ** sirve para calcular la potencia del operando de la izquierda
//elevado al de la derecha.
return numero**2;}
//Una función JavaScript se ejecuta cuando "algo" lo invoca (lo llama).
console.log("La potencia de 4 es: "+cuadrado(4));
```

.js sintaxis básica: objetos

```
// Un objeto en JavaScript es una lista de pares "clave:valor" separadas por , dentro de {}
// Se pueden crear de diferentes maneras
const persona = {
        nombre:"Juan",
        apellido:"Nadie",
        edad:25
};
// También podríamos crearlo así
const persona2 = {}
 // Y añado sus propiedades después:
persona2.nombre="Fermín";
persona2.apellido="Trujillo";
persona2.edad=33;
persona.colorDeOjos="Verdes";
// O creando una instancia de "Object"
const persona2 = new Object();
// Y luego rellenando sus propiedades:
persona3.nombre="Antonio";
persona3.primerApellido="Recío";
persona3.segundoApellido="Matamoros";
```

.js sintaxis básica: objetos

```
// También podemos crear un constructor
function Persona(nombre, primerApellido,segundoApellido){
        this.nombre = nombre;
        this.primerApellido = primerApellido;
        this.segundoApellido = segundoApellido;
}
// Para luego crear un objeto utilizando dicho constructor
const persona4 = new Persona("Amador","Rivas","Borderline");
// Podemos acceder a las propiedades de un objeto de 2 maneras diferentes:
console.log("1ª Persona: "+persona.nombre+" "+persona["apellido"]);
console.log("2ª Persona: "+persona2.nombre+" "+persona2.apellido);
console.log("3ª Persona: "+persona3["nombre"]+" "+persona3["primerApellido"]+"
"+persona3["segundoApellido"]);
// También podemos eliminar propiedades de un objeto
console.log("Edad 1ª persona: "+persona.edad);delete persona.edad;
// Si devuelve "undefined" la condición se evalua a "false"
// El siguiente "if" también se podría escribir así:
// if (typeof(persona.edad)!="undefined")
if (persona.edad) {
        console.log("Edad 1ª persona: "+persona.edad);
} else{
        console.log("Se eliminó la propiedad \"edad\" de la 1ª persona.");
}
```

.js sintaxis básica: objetos

```
// Por último también podremos añadir métodos a nuestros objetos:
const persona5 = {
        nombre: "Javier",
        apellido : "Maroto",
        edad : 32,
        nombreCompleto : function() {
                return this.nombre + " " + this.apellido;
        },
        indicarEdad(){
                return this.edad + " años"; }
};
// Y por supuesto llamar a dichos métodos
console.log("5ª Persona: "+persona5.nombreCompleto());
console.log("Y tiene "+persona5.indicarEdad());
```

Selección y Acceso a objetos del BOM y DOM

```
/*Podemos acceder a las propiedades de un objeto de 3 maneras diferentes:
Ya vistas en objetos:
 Notación de punto: nombreVariableObjeto.nombrePropiedad
 Notación de corchetes nombreVariableObjeto["nombrePropiedad"]

Acceso dinámico con una variable cuyo valor coincide con la propiedad
del objeto variable = "nombrePropiedad" y utilizo objeto[variable]*/
```

JS notacionpuntocorchete.js > ...

```
1  let alumno = {
2      nombre: "Carlos",
3      "nota final": 8.5
4  };
5
6  // Notación punto
7  console.log("notación punto alumno.nombre; ", alumno.nombre);      // Carlos
8
9  // Notación corchetes
10 console.log("notación corchete alumno[nota final]: , ",  alumno.["nota final"]); // 8.5
11
12 // Acceso dinámico
13 let clave = "nombre";
14 console.log("acceso usando la variable clave: ",  alumno[clave]);       // Carlos
```

Selección y Acceso a objetos del BOM y DOM

En este apartado vamos a ver cómo acceder a las propiedades y objetos que forman parte del:

- **BOM** (Browser Object Model) → representa el navegador
- **DOM** (Document Object Model) → representa el documento HTML

Tanto el BOM como el DOM se componen de **objetos**, que a su vez contienen propiedades y métodos.

Podemos acceder a todos ellos **para obtener información** (leer valores) **o modificar información** (cambiar el DOM, manejar eventos, etc.).

Selección y Acceso a objetos: BOM

El BOM está asociado al objeto *window*, que representa la ventana del navegador.

Todas las funciones globales, variables globales y objetos globales de JavaScript se convierten en propiedades del objeto window. Por eso podemos acceder a muchas cosas de dos maneras diferentes:

```
// Accedemos al objeto "alert" que es una propiedad del objeto "window"
window.alert("Hola");   // Forma explícita, usando el objeto window
// Otra manera de acceder al objeto "alert", directamente llamando a la propiedad de "window"
// pero sin hacer referencia al objeto "window" ya que todas sus propiedades y funciones son
// consideradas globales.
alert("Hola");          // Forma implícita, window está oculto

// Otro ejemplo accedemos al objeto "document" que es una propiedad del objeto "window"
window.document;
document;
```

Selección y Acceso a objetos: BOM

Tarea en Aula Virtual para explorar las principales propiedades del BOM

Selección y Acceso a objetos: DOM

El DOM es representado por el objeto *document*, que representa toda la página web, estructurando sus elementos como una jerarquía de nodos (elementos HTML, texto o comentarios). Para acceder a los diferentes elementos del document, podremos utilizar alguno de sus métodos de selección de nodos:

Método	Descripción
document.getElementById(id)	Encuentra un elemento por el valor de su atributo **Id** (pasado por parámetro como un String)
document.getElementsByTagName(name)	Devuelve una colección de elementos (se recorren como un Array empezando por la posición 0) correspondientes con el **nombre de la etiqueta** HTML
document.getElementsByClassName(name)	Devuelve una colección de elementos (se recorren como un Array empezando por la posición 0) correspondientes con los elementos HTML que tengan el **atributo class** con el valor pasado por parámetro en forma de String)
document.getElementsByName(name)	Devuelve una lista de nodos (se recorren como un array empezando por la posición 0)c orrespondientes con los elementos **HTML** que tengan el **atributo name** con el valor pasado por parámetro en forma de String.

Selección y Acceso a objetos: DOM

Método	Descripción
document.querySelector(selectorsCSS)	Devuelve el **primer** elemento que coincida con el selector **CSS** pasado como parámetro en forma de String
document.querySelectorAll(selectorsCSS)	Devuelve una lista de nodos (se recorren como un array empezando por la posición 0) correspondientes con los elementos **HTML** que coincidan con el selector **CSS** (o selectores CSS separados por comas en el String) pasado por parámetro en forma de String.

Selección y Acceso a objetos: Ejemplo

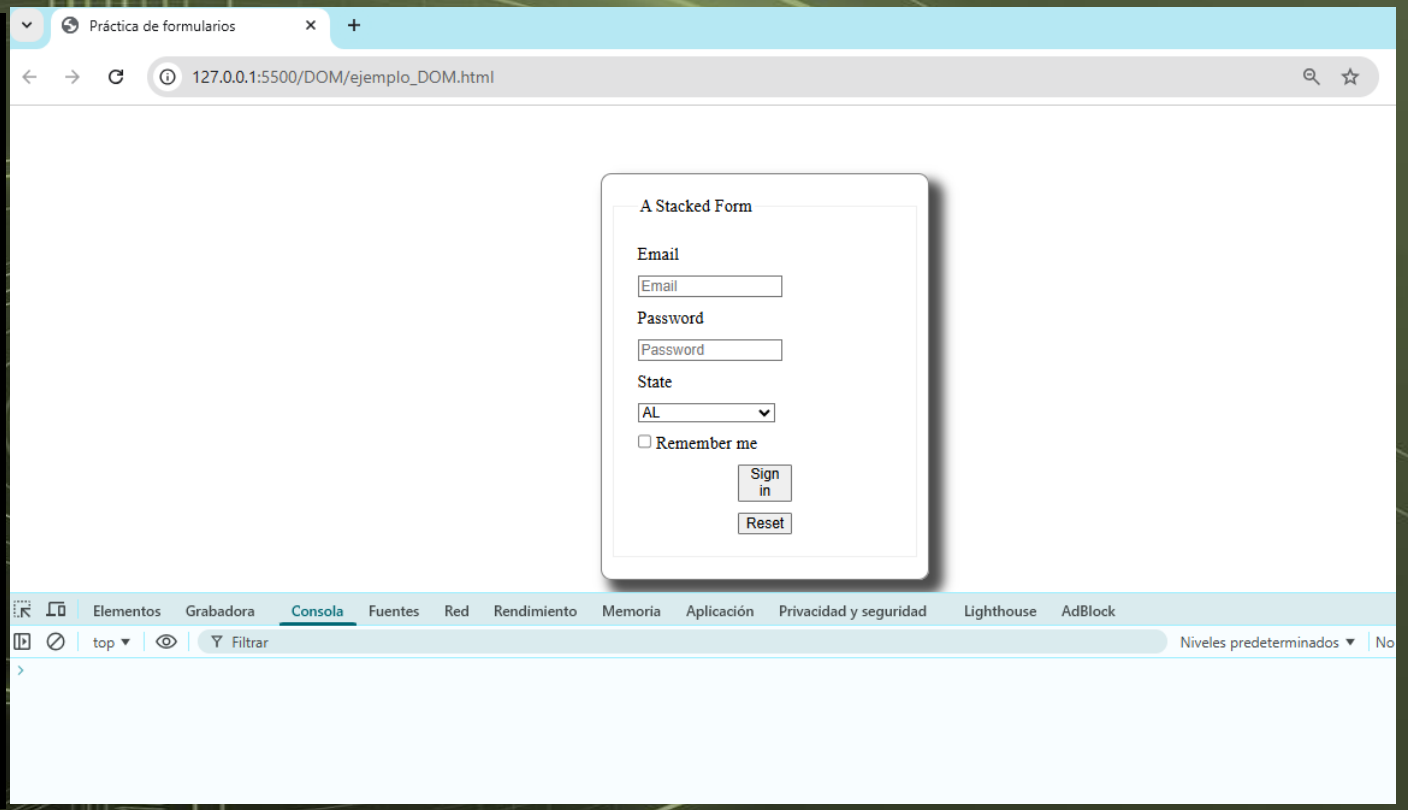
```html
<body>
    <div class="contenedor">
        <div class="formulario">
            <form>
                <fieldset>
                    <legend>A Stacked Form</legend>
                    <label for="stacked-email1">Email</label>
                    <input type="email" name="email" id="stacked-email1" placeholder="Email" />
                    <span></span>
                    <label for="stacked-password">Password</label>
                    <input type="password" id="stacked-password" name="pass" placeholder="Password" />
                    <span></span>
                    <label for="stacked-state">State</label>
                    <select id="stacked-state" name="estado">
                        <option>AL</option>
                        <option>CA</option>
                        <option>IL</option>
                    </select>
                    <span></span>
                    <label for="stacked-remember">
                        <input type="checkbox" id="stacked-remember" name="opcion" /> Remember me
                    </label>
                    <button type="submit" class="pure-button pure-button-primary">Sign in</button>
                    <button type="reset">Reset</button>
                </fieldset>
            </form>
        </div>
    </div>
</body>
```

OM > # formularios_validacion.css > ...

```css
* {
    margin: 10px auto;
}
.contenedor {
    display: flex;
    flex-flow: row wrap;
}

legend {
    margin: 10px 10px;
}

.formulario {
    border: 1px solid gray;
    width: 20%;
    padding: 10px;
    border-radius: 10px;
    box-shadow: 10px 10px 10px 3px rgba(1,1,1,0.7);
    margin-top: 3%;
    text-align: left;
}

/* Estilos para el formulario 1:
1) Todos los label deben de estar alienados verticalmente con sus respectivos controles de entrada
2) Todos los controles de entrada deben ser del mismo tamaño
3) Tanto los labels como los inputs deben estar alineados a la izquierda
4) El span de al lado del campo Email se mostrará cuándo se vaya a rellenar dicho campo en un color de texto rojo, más pequeño que el resto de textos.
5) El botón debe de estar centrado*/

label,input,select {
    display:block;
    width: 50%;
    margin: 10px;
}

select + label,label > input {
    display: inline-block;
    width: auto;
}

label > input {
    display: inline;
    margin: 0;
}

span {
    font-size: 0.7rem;
    margin:10px;
    display: none;
    color: red;
}

button {
    display: block;
    text-align: center;
    width: 20%;
    margin: 10px auto;
}
```

Selección y Acceso a objetos: Ejemplo

```html
<body>
    <div class="contenedor">
        <div class="formulario">
            <form>
                <fieldset>
                    <legend>A Stacked Form</legend>
                    <label for="stacked-email1">Email</label>
                    <input type="email" name="email" id="stacked-email1" placeholder="Email" />
                    <span></span>
                    <label for="stacked-password">Password</label>
                    <input type="password" id="stacked-password" name="pass" placeholder="Password" />
                    <span></span>
                    <label for="stacked-state">State</label>
                    <select id="stacked-state" name="estado">
                        <option>AL</option>
                        <option>CA</option>
                        <option>IL</option>
                    </select>
                    <span></span>
                    <label for="stacked-remember">
                        <input type="checkbox" id="stacked-remember" name="opcion" /> Remember me
                    </label>
                    <button type="submit" class="pure-button pure-button-primary">Sign in</button>
                    <button type="reset">Reset</button>
                </fieldset>
            </form>
        </div>
    </div>
</body>
```



Selección y Acceso a objetos: Ejemplo

```html
<body>
    <div class="contenedor">
        <div class="formulario">
            <form>
                <fieldset>
                    <legend>A Stacked Form</legend>
                    <label for="stacked-email1">Email</label>
                    <input type="email" name="email" id="stacked-email1" placeholder="Email" />
                    <span></span>
                    <label for="stacked-password">Password</label>
                    <input type="password" id="stacked-password" name="pass" placeholder="Password" />
                    <span></span>
                    <label for="stacked-state">State</label>
                    <select id="stacked-state" name="estado">
                        <option>AL</option>
                        <option>CA</option>
                        <option>IL</option>
                    </select>
                    <span></span>
                    <label for="stacked-remember">
                        <input type="checkbox" id="stacked-remember" name="opcion" /> Remember me
                    </label>
                    <button type="submit" class="pure-button pure-button-primary">Sign in</button>
                    <button type="reset">Reset</button>
                </fieldset>
            </form>
        </div>
    </div>
</body>
```

Obtener un elemento por su id:

```
> console.log(  "Elemento con id='stacked-state':\n",  document.getElementById("stacked-state").outerHTML)

Elemento con id='stacked-state':
 <select id="stacked-state" name="estado">
                            <option>AL</option>
                            <option>CA</option>
                            <option>IL</option>
                        </select>
```

Obtener el primer <input> del documento:

```
> console.log(  "Primer <input> del formulario:\n",
    document.getElementsByTagName("input")[0].outerHTML
  )

  Primer <input> del formulario:
   <input type="email" name="email" id="stacked-email1" placeholder="Email">
```

Obtener elemento por clase (class="pure-button")

```
> console.log( "Primer elemento con class='pure-button':\n",document.getElementsByClassName("pure-button")[0].outerHTML)

  Primer elemento con class='pure-button':
   <button type="submit" class="pure-button pure-button-primary">Sign in</button>
```

Selección y Acceso a objetos: Ejemplo

```html
<body>
    <div class="contenedor">
        <div class="formulario">
            <form>
                <fieldset>
                    <legend>A Stacked Form</legend>
                    <label for="stacked-email1">Email</label>
                    <input type="email" name="email" id="stacked-email1" placeholder="Email" />
                    <span></span>
                    <label for="stacked-password">Password</label>
                    <input type="password" id="stacked-password" name="pass" placeholder="Password" />
                    <span></span>
                    <label for="stacked-state">State</label>
                    <select id="stacked-state" name="estado">
                        <option>AL</option>
                        <option>CA</option>
                        <option>IL</option>
                    </select>
                    <span></span>
                    <label for="stacked-remember">
                        <input type="checkbox" id="stacked-remember" name="opcion" /> Remember me
                    </label>
                    <button type="submit" class="pure-button pure-button-primary">Sign in</button>
                    <button type="reset">Reset</button>
                </fieldset>
            </form>
        </div>
    </div>
</body>
```

Obtener un elemento por atributo name:

```
> console.log("Primer elemento con name='pass':\n",
    document.getElementsByName("pass")[0].outerHTML)

Primer elemento con name='pass':
 <input type="password" id="stacked-password" name="pass" placeholder="Password">
```

```
> console.log(  "Valor introducido en name='pass':\n",
    document.getElementsByName("pass")[0].value)

Valor introducido en name='pass':
 1234
```

Obtener la última <option> de la selección del "State"

```
> console.log(  "Última opción del select 'State':\n",
    document.querySelector("[name='estado'] > option:last-child").outerHTML)

Última opción del select 'State':
 <option>IL</option>
```

Obtener TODAS las opciones del "State"

```
> for (let i=0, lista=document.querySelectorAll("[name='estado'] > option"); i<lista.length; i++){
        console.log(lista[i].outerHTML);
    }
<option>AL</option>
<option>CA</option>
<option>IL</option>
```

Creación y Modificación de objetos

Hay una forma segura y recomendada para **crear** y **agregar** elementos dinámicamente en el DOM con JavaScript, mediante la combinación de los métodos ***createElement*** y ***appendChild*** del objeto document

Método	Descripción
parentElement.appendChild(node)	**Agrega** un **nodo** (elemento) como el **último hijo** de un elemento.
document.createElement(type)	**Crea un nodo de elemento.**

Pasos para crear y añadir un elemento al DOM

1. Seleccionar el elemento padre
   Obtenemos el elemento del DOM que actuará como contenedor o padre del nuevo elemento que queremos insertar.
2. Crear el nuevo elemento HTML
   Utilizando el método document.createElement().
3. Configurar el nuevo elemento. Antes de insertarlo en el DOM, podemos:
   - Asignarle texto (textContent)
   - Añadir clases (classList.add)
   - Establecer atributos (setAttribute)
   - Añadir estilos, eventos, etc.
4. Insertar el elemento en el DOM como hijo del elemento padre utilizando appendChild()

Creación y Modificación de objetos

Crear un párrafo

EducaMadrid - Plataforma Educativa. Comunidad de Madrid - Ayuda

Este párrafo ha sido creado desde JavaScript.

Elementos | Grabadora | Consola | Fuentes | Red | Rendimiento | Memoria

top ▾ | Filtrar

```
> let p = document.createElement("p");
  p.textContent = "Este párrafo ha sido creado desde JavaScript.";
  document.body.appendChild(p);
<· <p>Este párrafo ha sido creado desde JavaScript.</p>
```

Crear un botón

EducaMadrid - Plataforma Educativa. Comunidad de Madrid - Ayuda

Púlsame

Elementos | Grabadora | Consola | Fuentes | Red | Rendimiento | Memoria

top ▾ | Filtrar

```
> let boton = document.createElement("button")
  boton.textContent = "Púlsame"
  boton.classList.add("btn")
  document.body.appendChild(boton)
<· <button class="btn">Púlsame</button>
```

Modificación de elementos: Propiedades y Atributos

Podemos modificar las propiedades y atributos de los elementos del DOM:

Métodos para la modificación de Propiedades	Descripción
element.textcontent	Devuelve el **contenido** de **texto** de un elemento. También se puede cambiar el contenido asignándole otro texto
element.innerHTML	Devuelve el **contenido** de este elemento HTML, lo que hay **entre la etiqueta de apertura y cierre**, en forma de texto
element.outerHTML	Devuelve el **elemento HTML, incluidos los atributos, la etiqueta de inicio y la etiqueta final**

Métodos para la modificación de Atributos	Descripción
element.hasAttribute("nombreDelAtributo") Ej: element.hasAttribute("target")	Devolverá un valor **boolean** que indica **si element** tiene el atributo. En el ejemplo, si element tiene el atributo *target*, devolverá *true*
element.getAttribute("nombreDelAtributo") Ej: element.getAttribute("class")	Devuelve el **valor** del atributo de este elemento HTML.
element.setAttribute("nombreDelAtributo","valor") Ej: element.hasAttribute("class","democlass")	En el ejemplo, asignará el valor "democlass" al atributo class. Si el atributo class no existe, lo creará con el valor "democlass".

Detección de eventos: addEventListener

JavaScript es un lenguaje orientado a eventos que ***permite*** desarrollar ***scripts*** (fragmentos de código) ***que ejecuten acciones en respuesta a un evento*** ocurrido en el navegador. Un evento en el navegador es una acción o suceso que ocurre dentro del contexto de una página web o aplicación web, y que puede ser detectado y gestionado Mediante JavaScript.

Para poder detectar un evento ocurrido en un objeto del DOM podemos utilizar el método ***addEventListener*** de cualquiera de los elementos:

Método	Descripción
element.addEventListener(“nombreDelEvento”,”funciónnQueEjecutaAlDetectarlo”) Ej: element.hasAttribute(“target”)	Agrega un **escuchador de eventos** a dicho **element**, que estará atento a detectar el evento que coincida con el "***nombreDelEvento***“ para que cuando lo detecte ejecutará la ***funcion***.

Detección de eventos

Eventos asociados a elementos (Casos comunes)

Elemento	Nombre del Evento	Descripción
window	**load**	Se produce cuando la página y todos sus recursos (como imágenes y scripts) se han cargado completamente..
window	**resize**	Se produce cuando se cambia el tamaño de la ventana del navegador.
window	**scroll**	Se produce cuando se ha desplazado el scroll de la página.
document	**DOMContentLoaded**	Se produce cuando el ***documento HTML*** está completamente cargado y el árbol DOM está construido, pero es posible que los recursos externos como <img> y hojas de estilo aún no se hayan cargado.
element	**click**	Se produce cuando el usuario hace clic en un elemento
element	**dblclick**	Se produce cuando el usuario hace doble clic en un elemento
element	**mouseover**	Se produce cuando el cursor del ratón pasa sobre un elemento.
element	**mouseout**	Se produce cuando el cursor del ratón sale de un elemento.

Detección de eventos

Elemento	Nombre del Evento	Descripción
element (que represente un formulario)	**submit**	Se produce cuando se envía un formulario
element (que represente un elemento al que se le cambia el atributo value)	**change**	Se produce cuando el valor de un campo del formulario cambia.
element (que represente un elemento al que se le puede establecer un valor al atributo value)	**input**	Se produce cuando el usuario introduce datos en un campo de formulario

Eliminación de elementos

Hay 2 maneras en las que podemos eliminar elementos del DOM (representado por el objeto document)

Métodos para la eliminación de elementos	Descripción
parentElement.removeChild(node) Ej: element.parentNode.removeChild(element)	Eliminará el elemento representado por el node de su nodo (elemento padre.
element.remove( ) Ej: let element = document.getElementById("demo"); element.remove();	Eliminará el elemento que llamó a la funcióndel documento (document).

DDINT – 2 DAM

Luis.delgado15@educa.madrid.org